



INSTITUTO PROFESIONAL SAN SEBASTIAN

Proyecto Backend ProviEmplea

Departamento de Empleo - Municipalidad de Providencia

Evaluación Sumativa U3 - Desarrollo Backend

Instituto Profesional San Sebastián

Profesor: Carlos Caro

Miguel Saldarriaga
Marcelo Salas

Indice de Contenidos

N	Seccion	Pagina
1	Introduccion al Proyecto	3
2	Estructura General del Proyecto	3
3	Archivo: conexion.php	4
4	Archivo: v1.php	5
5	Archivo: docs.php	6
6	Modulo CANDIDATOS	6
6.1	CANDIDATOS/index.php	6
6.2	CANDIDATOS/modeloCandidato.php	9
7	Modulo EMPRESAS	11
7.1	EMPRESAS/index.php	11
7.2	EMPRESAS/modeloEmpresa.php	13
8	Modulo HABILIDADES	13
8.1	HABILIDADES/index.php	13
8.2	HABILIDADES/modeloHabilidad.php	13
9	Documentacion Swagger (swagger.yaml)	15
9.1	Estructura del archivo	15
9.2	Bloque info y servers	16
9.3	Bloque components	17
9.3	Bloque paths - Endpoints documentados	19
9.4	Codigos de respuesta HTTP	21
9.5	Como usar Swagger	22
10	Capturas de Pantalla - Swagger UI	23
11	Capturas de Pantalla - Postman	32
12	Base de Datos - Estructura de tablas	37
13	Guia de instalacion y puesta en marcha	39

1. Introduccion al Proyecto

ProviEmplea es una plataforma digital de busqueda inversa de empleo desarrollada para el Departamento de Empleo de la Municipalidad de Providencia. A diferencia de las plataformas tradicionales, son las empresas quienes buscan activamente a los candidatos y no al revés.

El presente documento describe el backend de la plataforma, desarrollado en PHP puro con base de datos MySQL y documentado mediante Swagger (OpenAPI 3.0). El backend expone una API REST con tres modulos principales:

- CANDIDATOS: perfiles laborales anonimos de vecinos de Providencia.
- EMPRESAS: registro de empresas habilitadas para buscar talento.
- HABILIDADES: catalogo de habilidades tecnicas y blandas de la plataforma.

Cada modulo implementa las cinco operaciones CRUD: GET, POST, PUT, PATCH y DELETE, protegidas mediante tokens Bearer unicos por metodo HTTP.

2. Estructura General del Proyecto

El proyecto se organiza dentro de la carpeta API/ con la siguiente estructura de archivos y directorios:

```
htdocs/proviemplea/  API/      conexion.php          <- Clase de conexion a MySQL
v1.php              <- Cabeceras CORS y validacion de token      docs.php
<- Redireccion a Swagger UI    swagger.yaml          <- Especificacion OpenAPI 3.0
CANDIDATOS/          index.php              <- Controlador CRUD de candidatos
modeloCandidato.php <- Modelo y consultas SQL de candidatos      EMPRESAS/
index.php            <- Controlador CRUD de empresas      modeloEmpresa.php    <-
Modelo y consultas SQL de empresas      HABILIDADES/        index.php            <-
Controlador CRUD de habilidades          modeloHabilidad.php  <- Modelo y consultas SQL
de habilidades      swagger-ui/    dist/                <- Archivos estaticos de
Swagger UI
```

3. Archivo: conexion.php

Este archivo define la clase Conexion, responsable de establecer y cerrar la conexion a la base de datos MySQL. Es utilizada por todos los modelos del sistema.

Ruta del archivo

API/conexion.php

Funcion de cada metodo

Metodo	Descripcion
__construct()	Inicializa los parametros de conexion: host (127.0.0.1), puerto (3306), nombre de la BD (proviemplea), usuario (root) y contrasena vacia para entorno local. Tambien almacena \$_SERVER["SERVER_NAME"].
getConnection()	Establece la conexion con mysqli_connect(), configura el charset a UTF-8 y retorna el objeto de conexion. Captura errores con try/catch y los registra en el log del servidor.
closeConnection()	Cierra la conexion activa con mysqli_close() para liberar recursos del servidor, solo si la conexion fue establecida previamente.

Codigo del archivo

```
<?php class Conexion {    private $connection;    private $host;    private $username;    private $password;    private $db;    private $port;    private $server;    public function __construct() {        $this->server = $_SERVER["SERVER_NAME"];        $this->connection = null;        $this->host = "127.0.0.1";        $this->port = "3306";        $this->db = "proviemplea";        $this->username = "root";        $this->password = "";    }    public function getConnection() {        try {            $this->connection = mysqli_connect($this->host, $this->username, $this->password, $this->db, $this->port);            mysqli_set_charset($this->connection, "utf8");            return $this->connection;        } catch (Exception $e) {            error_log($e->getMessage());            die("Error al conectar a la BD: " . $e->getMessage());        }    }    public function closeConnection() {        if ($this->connection) {            mysqli_close($this->connection);        }    } }
```

4. Archivo: v1.php

Este archivo es el nucleo de seguridad y configuracion de la API. Se incluye en el index.php de cada modulo como primer paso antes de ejecutar cualquier logica de negocio.

Ruta del archivo

```
API/v1.php
```

Responsabilidades del archivo

- Detectar el metodo HTTP de la solicitud entrante (\$_SERVER["REQUEST_METHOD"]) y almacenarlo en la variable global \$_method.
- Configurar los headers CORS para permitir solicitudes desde cualquier origen (Access-Control-Allow-Origin: *), aceptando los metodos GET, POST, PUT, PATCH y DELETE.
- Responder inmediatamente con codigo 200 ante solicitudes OPTIONS (preflight de CORS) para que los navegadores y clientes como Postman puedan verificar los permisos.
- Leer el header Authorization de la solicitud usando getallheaders() y almacenar su valor en la variable global \$_authorization.
- Si el header Authorization no existe, retornar inmediatamente un codigo 401 (Sin Autorizacion) y detener la ejecucion con exit.
- Si ocurre cualquier excepcion durante la lectura del header, retornar un codigo 500 con el mensaje de error interno.

Estructura general del archivo

```
<?php global $_method, $_authorization; // 1. Captura el verbo HTTP de la solicitud
$_method = $_SERVER["REQUEST_METHOD"]; // 2. Cabeceras CORS - permite acceso desde
cualquier origen header("Access-Control-Allow-Origin: *"); header("Access-Control-
Allow-Methods: GET, POST, PUT, PATCH, DELETE"); header("Access-Control-Allow-
Headers: Authorization, Content-Type"); header("Content-Type: application/json");
// 3. Responde preflight OPTIONS de inmediato if ($_method === "OPTIONS")
{ http_response_code(200); exit; } // 4. Lee y valida el token Bearer try
{
    $headers = getallheaders();    if (!isset($headers["Authorization"]))
    {
        http_response_code(401);    echo
json_encode(["type"=>"error", "msg"=>"Sin Autorizacion"]);    exit;    }
$_authorization = $headers["Authorization"]; } catch (Exception $e)
{
    http_response_code(500);    echo json_encode(["type"=>"error", "msg"=>$e-
>getMessage()]);    exit; }
```

5. Archivo: docs.php

Archivo simple de una sola instruccion. Su proposito es redirigir al explorador visual de la API (Swagger UI) cuando el usuario accede a la ruta /API/docs.php en el navegador.

Ruta del archivo

```
API/docs.php
```

Codigo completo

```
<?php // docs.php - Redirige al explorador visual de la API (Swagger UI) // El
archivo swagger.yaml debe estar en la carpeta API/ // La carpeta swagger-ui debe
estar dentro de API/ header("Location: ../swagger-
ui/dist/index.html?url=../../swagger.yaml"); exit; ?>
```

Utiliza header("Location: ...") de PHP para redirigir al archivo index.html de Swagger UI, pasando como parametro url la ruta relativa al swagger.yaml. Esto permite que cualquier persona acceda a la documentacion interactiva navegando a:

```
http://localhost/proviemplea/API/docs.php
```

6. Modulo CANDIDATOS

Modulo central de la plataforma. Gestiona los perfiles laborales de los vecinos de Providencia que buscan empleo. Por diseno, los perfiles NO incluyen datos sociodemograficos (nombre, edad, genero, comuna) para evitar discriminacion arbitraria en el proceso de seleccion, cumpliendo con el principio de curriculum ciego del proyecto.

6.1 CANDIDATOS/index.php

Controlador que recibe todas las solicitudes HTTP al endpoint /CANDIDATOS/ y las redirige a la operacion CRUD correspondiente segun el metodo de la solicitud.

Archivos incluidos al inicio

- ../v1.php - cabeceras CORS y validacion del token Bearer
- ../conexion.php - clase de conexion a la base de datos
- modeloCandidato.php - clase con las operaciones SQL

Tokens Bearer por metodo HTTP

Metodo	Token Bearer requerido	Operacion
GET	Bearer candidatos.get	Obtener lista de candidatos
POST	Bearer candidatos.post	Crear nuevo candidato
PUT	Bearer candidatos.put	Actualizar candidato completo
PATCH	Bearer candidatos.patch	Cambiar estado activo/inactivo
DELETE	Bearer candidatos.delete	Eliminar candidato por ID

Descripcion de cada operacion

Metodo	Validacion	Codigo exito	Respuesta
GET	Solo verifica token	200	{"data": [...lista de candidatos...]}
POST	VCH_TITULO_PERFIL, VCH_DESCRIPCION, INT_AÑIOS_EXPERIENCIA	201	{"type":"msg","msg":"Candidato creado correctamente"}
PUT	ID_CANDIDATO + los 3 campos del POST	200	{"type":"msg","msg":"Candidato actualizado correctamente"}
PATCH	Solo ID_CANDIDATO en el body	200	{"type":"msg","msg":"Estado del candidato actualizado"}
DELETE	Solo ID_CANDIDATO en el body	200	{"type":"msg","msg":"Candidato eliminado correctamente"}

Codigos de error comunes

Codigo	Cuando ocurre
400	Faltan campos requeridos en el body del request
401	No se envio el header Authorization
403	El token es incorrecto para ese metodo HTTP
422	El query SQL se ejecuto pero no afecto ninguna fila
501	Se uso un metodo HTTP no soportado (ej: CONNECT, HEAD)

Fragmento clave: estructura del switch

```
switch ($_method) {      case "GET":          if ($_authorization === "Bearer
candidatos.get") {      $modelo = new modeloCandidato();
http_response_code(200);      echo json_encode(["data" => $modelo-
>getAll()]);      } else {          http_response_code(403);          echo
json_encode(["type"=>"error","msg"=>"Acceso Prohibido"]);      }      break;
case "POST":          if ($_authorization === "Bearer candidatos.post")
{          $body = json_decode(file_get_contents("php://input", true));
if (!isset($body->VCH_TITULO_PERFIL) || !isset($body->VCH_DESCRIPCION)
|| !isset($body->INT_AÑIOS_EXPERIENCIA)) {          http_response_code(400);
echo json_encode(["type"=>"error","msg"=>"Datos incompletos"]);
break;          }          // ... instanciar modelo, setters, add(),
respuesta ...      }      break;          // ... PUT, DELETE, PATCH, default ... }
```

6.2 CANDIDATOS/modeloCandidato.php

Clase que implementa todas las consultas SQL del modulo de candidatos. Aplica el patron de acceso a datos (DAO) y utiliza Prepared Statements en todas las operaciones de escritura para prevenir inyeccion SQL.

Propiedades de la clase

Propiedad	Tipo	Columna en BD	Descripcion
\$ID_CANDIDATO	int	ID_CANDIDATO	Identificador unico autoincremental
\$VCH_TITULO_PERFIL	string	VCH_TITULO_PERFIL	Titulo o cargo del perfil (sin datos personales)
\$VCH_DESCRIPCION	string	VCH_DESCRIPCION	Descripcion de habilidades y experiencia
\$INT_ANIOS_EXPERIENCIA	int	INT_ANIOS_EXPERIENCIA	Anos de experiencia laboral
\$INT_ACTIVO	int	INT_ACTIVO	Estado: 1 = visible para empresas, 0 = oculto

Metodos CRUD

Metodo	Operacion SQL	Descripcion
getAll()	SELECT	Retorna todos los candidatos de tbl_candidato como un array asociativo. No filtra por activo para que el administrador vea todos los perfiles.
add(\$nuevo)	INSERT	Inserta un nuevo candidato con INT_ACTIVO = 1 (activo por defecto). Usa prepared statement con bind_param tipo "ssi" (string, string, int).
update(\$nuevo)	UPDATE	Actualiza los tres campos del perfil identificando el registro por ID_CANDIDATO. Prepared statement tipo "ssii".
delete(\$nuevo)	DELETE	Elimina el registro definitivamente de la BD por ID_CANDIDATO. Prepared statement tipo "i".
toggleActivo()	UPDATE	Usa la expresion SQL IF(INT_ACTIVO=1,0,1) para invertir el estado en una sola consulta sin necesidad de leer el valor actual.

Fragmentos clave del modelo

```
// getAll() - bucle de construccion del array de respuesta public function getAll()
{
    $lista = [];
    $con = new Conexion();
    $query = "SELECT ID_CANDIDATO,
VCH_TITULO_PERFIL, VCH_DESCRIPCION,
INT_ANIOS_EXPERIENCIA, INT_ACTIVO
FROM tbl_candidato";
    $rs = mysqli_query($con->getConnection(), $query);
    if ($rs) {
        while ($registro = mysqli_fetch_assoc($rs)) {
            $stupla = new modeloCandidato();
            // setters ...
            array_push($lista, [
                "id" => $stupla->getID_CANDIDATO(),
                "titulo_perfil" => $stupla->getVCH_TITULO_PERFIL(),
                "descripcion" => $stupla->getVCH_DESCRIPCION(),
                "anios_experiencia" => $stupla->getINT_ANIOS_EXPERIENCIA(),
                "activo" => $stupla->getINT_ACTIVO()
            ]);
            mysqli_free_result($rs);
        }
        $con->closeConnection();
        return $lista;
    }
    // add() - INSERT con prepared statement
    $stmt = mysqli_prepare($con->getConnection(), "INSERT INTO
tbl_candidato (VCH_TITULO_PERFIL, VCH_DESCRIPCION, INT_ANIOS_EXPERIENCIA,
INT_ACTIVO) VALUES (?, ?, ?, 1)");
    mysqli_stmt_bind_param($stmt, "ssi",
    $_nuevo->getVCH_TITULO_PERFIL(),
    $_nuevo->getVCH_DESCRIPCION(),
    $_nuevo->getINT_ANIOS_EXPERIENCIA());
    // toggleActivo() - cambio de estado sin doble query
    $stmt = mysqli_prepare($con->getConnection(), "UPDATE tbl_candidato SET
INT_ACTIVO = IF(INT_ACTIVO = 1, 0, 1) WHERE ID_CANDIDATO = ?");
    mysqli_stmt_bind_param($stmt, "i", $_nuevo->getID_CANDIDATO());
}
```

7. Modulo EMPRESAS

Gestiona el registro de empresas habilitadas para buscar y contactar candidatos en la plataforma. Las empresas pueden ser de tipo contratacion directa, EST (Empresa de Servicios Transitorios) u outsourcing.

7.1 EMPRESAS/index.php

Controlador identico en estructura al de CANDIDATOS, pero para el recurso empresa. Maneja el mismo patron de switch por metodo HTTP con tokens Bearer propios del modulo.

Tokens Bearer por metodo HTTP

Metodo	Token Bearer requerido	Operacion
GET	Bearer empresas.get	Obtener lista de empresas
POST	Bearer empresas.post	Registrar nueva empresa
PUT	Bearer empresas.put	Actualizar empresa completa
PATCH	Bearer empresas.patch	Cambiar estado activo/inactivo
DELETE	Bearer empresas.delete	Eliminar empresa por ID

Campos requeridos por operacion

Metodo	Campos obligatorios en el body
GET	(ninguno - solo token)
POST	VCH_NOMBRE_EMPRESA, VCH_RUBRO, VCH_DESCRIPCION
PUT	ID_EMPRESA, VCH_NOMBRE_EMPRESA, VCH_RUBRO, VCH_DESCRIPCION
PATCH	ID_EMPRESA
DELETE	ID_EMPRESA

7.2 EMPRESAS/modeloEmpresa.php

Clase de acceso a datos para tbl_empresa. Misma arquitectura que modeloCandidato con getters, setters y los cinco metodos CRUD usando Prepared Statements.

Propiedades de la clase

Propiedad	Tipo	Columna en BD
\$ID_EMPRESA	int	ID_EMPRESA
\$VCH_NOMBRE_EMPRESA	string	VCH_NOMBRE_EMPRESA
\$VCH_RUBRO	string	VCH_RUBRO
\$VCH_DESCRIPCION	string	VCH_DESCRIPCION
\$INT_ACTIVO	int	INT_ACTIVO

Diferencias clave respecto a modeloCandidato

- Metodo add(): INSERT con bind_param tipo "sss" (tres strings: nombre empresa, rubro y descripcion).
- Metodo update(): UPDATE con bind_param tipo "sssi" (tres strings + un int para el ID_EMPRESA).
- Metodo toggleActivo(): mismo patron IF(INT_ACTIVO=1,0,1) aplicado a tbl_empresa.
- El campo VCH_RUBRO permite categorizar la empresa segun su sector economico (tecnologia, salud, construccion, etc.).

8. Modulo HABILIDADES

Catalogo de habilidades disponibles en la plataforma. Se clasifican en dos categorias: Tecnica (conocimientos especificos como PHP, MySQL, Excel) y Blanda (competencias transversales como liderazgo, trabajo en equipo). Los candidatos asocian habilidades de este catalogo a su perfil.

8.1 HABILIDADES/index.php

Tokens Bearer por metodo HTTP

Metodo	Token Bearer requerido	Operacion
GET	Bearer habilidades.get	Obtener catalogo de habilidades
POST	Bearer habilidades.post	Agregar habilidad al catalogo
PUT	Bearer habilidades.put	Actualizar habilidad del catalogo
PATCH	Bearer habilidades.patch	Activar o desactivar habilidad
DELETE	Bearer habilidades.delete	Eliminar habilidad del catalogo

Campos requeridos por operacion

Metodo	Campos obligatorios en el body
GET	(ninguno - solo token)
POST	VCH_NOMBRE_HABILIDAD, VCH_CATEGORIA
PUT	ID_HABILIDAD, VCH_NOMBRE_HABILIDAD, VCH_CATEGORIA
PATCH	ID_HABILIDAD
DELETE	ID_HABILIDAD

8.2 HABILIDADES/modeloHabilidad.php

Propiedad	Tipo	Columna en BD
\$ID_HABILIDAD	int	ID_HABILIDAD
\$VCH_NOMBRE_HABILIDAD	string	VCH_NOMBRE_HABILIDAD
\$VCH_CATEGORIA	string	VCH_CATEGORIA (valores: Tecnica / Blanda)
\$INT_ACTIVO	int	INT_ACTIVO

Diferencias con los otros modulos

- El metodo add() usa bind_param tipo "ss" (solo dos strings: nombre y categoria).
- El metodo update() usa bind_param tipo "ssi" (dos strings + un int para el ID).
- El campo VCH_CATEGORIA actua como enumerado: solo acepta los valores "Tecnica" o "Blanda". Esta restriccion esta documentada en el swagger.yaml.
- Las habilidades inactivas (INT_ACTIVADO=0) no aparecen en el formulario de registro de candidatos.

9. Documentacion Swagger (swagger.yaml)

El archivo swagger.yaml es la especificacion completa de la API REST de ProviEmplea, escrita bajo el estandar OpenAPI 3.0.0. Este estandar es mundialmente reconocido para documentar APIs HTTP y es compatible con herramientas como Swagger UI, Postman, Insomnia, Redoc y generadores de codigo automatico.

El proposito principal del archivo es separar la documentacion del codigo. En lugar de escribir comentarios dentro del PHP, toda la descripcion de los endpoints, sus parametros, sus posibles respuestas y sus ejemplos de datos vive en este unico archivo YAML.

Ruta del archivo en el proyecto

```
htdocs/proviemplea/API/swagger.yaml
```

Resumen de secciones del archivo

Bloque	Lineas aprox.	Proposito
openapi:	1	Declara la version del estandar OpenAPI (3.0.0).
info:	30	Metadatos visibles en Swagger UI: titulo, version, descripcion con reglas de autenticacion, rate limiting y cache.
servers:	4	URL base del servidor. Swagger construye todas las URLs de prueba concatenando esta base con cada path.
components:	200	Definicion de schemas de datos reutilizables y esquemas de seguridad. Evita duplicar codigo entre endpoints.
paths:	460	El cuerpo principal. Define los 15 endpoints de la API con sus parametros, ejemplos y respuestas.

9.1 Bloque info: metadatos y politicas de la API

La seccion info es lo primero que ve el usuario al abrir Swagger UI. Aparece como encabezado en la parte superior con el titulo, la version y la descripcion completa de la API.

Sistema de autenticacion documentado

La descripcion explica que esta API asigna un token distinto por cada recurso Y por cada metodo HTTP. El mismo desarrollador que puede leer candidatos (candidatos.get) no puede crearlos a menos que tambien tenga el token candidatos.post. Este diseno permite un control de acceso granular por operacion.

```
info:  title: API ProviEmplea  version: 1.0.0  description: |  ##
Autenticacion  Cada recurso utiliza tokens Bearer distintos por metodo HTTP:
- GET  -> Bearer {recurso}.get  - POST  -> Bearer {recurso}.post  - PUT
-> Bearer {recurso}.put  - DELETE -> Bearer {recurso}.delete  - PATCH  ->
Bearer {recurso}.patch
```

Politica de Rate Limiting documentada

Tipo de operacion	Limite	Codigo de error si se supera
GET (lectura)	Maximo 100 solicitudes por minuto por IP	429 Too Many Requests
POST / PUT / PATCH / DELETE (escritura)	Maximo 30 solicitudes por minuto por IP	429 Too Many Requests

Bloque servers: URL base del servidor

```
servers:  - url: http://localhost/proviemplea/API  description: Servidor local
XAMPP (desarrollo)
```

Cuando el usuario hace clic en "Execute" en Swagger UI, la herramienta construye la URL completa concatenando esta base con el path del endpoint. Por ejemplo, para /CANDIDATOS/ la URL resultante es:

```
http://localhost/proviemplea/API/CANDIDATOS/
```

9.2 Bloque components: esquemas reutilizables

La seccion components permite definir estructuras de datos una sola vez y referenciarlas desde cualquier endpoint con la notacion \$ref. Esto elimina la duplicacion de codigo en el YAML.

9.2.1 securitySchemes: esquema de autenticacion

```
components:      securitySchemes:      bearerAuth:      type: http      scheme: bearer
bearerFormat: Token
```

Cuando un endpoint declara security: [{bearerAuth: []}], Swagger UI activa el boton "Authorize" permitiendo al usuario ingresar el token una sola vez y que se aplique a todas las solicitudes de prueba.

9.2.2 schemas: modelos de datos (11 schemas en total)

Grupo 1: schemas de CANDIDATOS

Schema	Tipo	Campos	Descripcion
CandidatoBody	object	3 requeridos	VCH_TITULO_PERFIL (string, max 100), VCH_DESCRIPCION (string, max 500), INT_ANIOS_EXPERIENCIA (integer, min 0). Todos required.
CandidatoResponse	object	5 campos	Estructura del GET: id, titulo_perfil, descripcion, anios_experiencia, activo (enum [0,1]).
CandidatoPut	allOf	4 requeridos	Hereda CandidatoBody y agrega ID_CANDIDATO (integer). Evita repetir los 3 campos del POST.

Grupo 2: schemas de EMPRESAS

Schema	Tipo	Campos	Descripcion
EmpresaBody	object	3 requeridos	VCH_NOMBRE_EMPRESA (string, max 100), VCH_RUBRO (string, max 100), VCH_DESCRIPCION (string, max 300).
EmpresaResponse	object	5 campos	Estructura del GET: id, nombre_empresa, rubro, descripcion, activo (enum [0,1]).
EmpresaPut	allOf	4 requeridos	Combina EmpresaBody con ID_EMPRESA (integer) para la operacion PUT.

Grupo 3: schemas de HABILIDADES

Schema	Tipo	Campos	Descripcion
HabilidadBody	object	2 requeridos	VCH_NOMBRE_HABILIDAD (string, max 100), VCH_CATEGORIA (string, enum ["Tecnica", "Blanda"]).
HabilidadResponse	object	4 campos	Estructura del GET: id, nombre_habilidad, categoria, activo.
HabilidadPut	allOf	3 requeridos	Combina HabilidadBody con ID_HABILIDAD (integer) para la operacion PUT.

Grupo 4: respuestas genericas reutilizables

Schema	Estructura JSON	Cuando se usa
RespuestaExito	{"type": "msg", "msg": "Texto descriptivo"}	Respuestas 200 y 201: confirma operacion exitosa.
RespuestaError	{"type": "error", "msg": "Descripcion del error"}	Respuestas 400, 401, 403, 422, 500: informa el tipo de error.

La directiva allOf: herencia de schemas

allOf permite que un schema herede las propiedades de otro y agregue campos adicionales, sin repetir la definicion completa. En ProviEmplea se usa en los schemas Put de los tres modulos:

```
CandidatoPut:  allOf:      - $ref: "#/components/schemas/CandidatoBody"  # hereda
los 3 campos del POST      - type: object                                required:      - ID_CANDIDATO
properties:      ID_CANDIDATO:      type: integer                        description: ID
del candidato a actualizar      example: 1
```

9.3 Bloque paths: documentacion de los 15 endpoints

La seccion paths es el corazon del swagger.yaml. Contiene la definicion completa de los 15 endpoints (3 modulos x 5 metodos HTTP). Cada endpoint se documenta bajo su URL relativa y dentro de ella se anidan los metodos HTTP soportados.

Campo YAML	Obligatorio	Descripcion
tags	Si	Agrupar el endpoint en Swagger UI. Se usan tres tags: Candidatos, Empresas y Habilidades, cada una como seccion colapsable.
summary	Si	Texto corto (una linea) que describe la operacion. Aparece en el listado antes de expandir el endpoint.
description	No	Texto largo con detalles adicionales en Markdown. Incluye informacion de cache y rate limiting por endpoint.
security	Si	Declara que el endpoint requiere autentificacion. Al referenciar bearerAuth, Swagger UI muestra el icono de candado.
requestBody	Condicional	Define el cuerpo de la solicitud para POST, PUT, PATCH y DELETE. Incluye schema y ejemplos predefinidos.
responses	Si	Lista todos los posibles codigos de respuesta HTTP con su descripcion, schema y ejemplo de JSON.

Ejemplo: endpoint GET /CANDIDATOS/

```
/CANDIDATOS/: get:      tags: [Candidatos]      summary: Obtener todos los
candidatos      description: |      Retorna la lista completa de candidatos.
Los perfiles NO incluyen datos sociodemograficos.      Cache: respuesta cacheable
por 60 segundos.      Rate limit: max 100 solicitudes/minuto.      security:      -
bearerAuth: []      responses:      "200":      description: Lista de candidatos
obtenida exitosamente      content:      application/json:
schema:      type: object      properties:      data:
type: array      items:      $ref:
"#/components/schemas/CandidatoResponse"
```

Ejemplo: endpoint POST con ejemplos multiples

```
post:      tags: [Candidatos]      summary: Crear un nuevo candidato      security:
- bearerAuth: []      requestBody:      required: true      content:
application/json:      schema:      $ref:
"#/components/schemas/CandidatoBody"      examples:      tecnico:
summary: Ejemplo perfil tecnico      value:
VCH_TITULO_PERFIL: "Tecnico en Informatica"      VCH_DESCRIPCION:
"Experiencia en soporte tecnico y redes"      INT_ANIOS_EXPERIENCIA: 3
contador:      summary: Ejemplo perfil contador      value:
VCH_TITULO_PERFIL: "Contador Auditor"      VCH_DESCRIPCION: "Manejo de
Excel avanzado y tributaria"      INT_ANIOS_EXPERIENCIA: 5
```

Ejemplo: endpoint PATCH (toggle activo)

```
patch:      summary: Encender o apagar un candidato (cambiar estado activo)
description: |      Cambia el estado de visibilidad del candidato.      Si esta
activo (1) lo desactiva (0), y viceversa.      Los candidatos inactivos no son
visibles para las empresas.      requestBody:      required: true      content:
application/json:      schema:      type: object      required:
[ID_CANDIDATO]      properties:      ID_CANDIDATO:
type: integer      example: 1
```

9.4 Codigos de respuesta HTTP: documentacion completa

Codigo HTTP	Nombre	Cuando ocurre en ProviEmplea	Respuesta JSON
200 OK	Exito	GET retorna datos. PUT, PATCH o DELETE se ejecutan sin errores.	{"data":[...]} o {"type":"msg","msg":"..."}
201 Created	Creado	El POST inserto exitosamente un nuevo registro en la base de datos.	{"type":"msg","msg":"Candidato creado correctamente"}
400 Bad Request	Solicitud erronea	El body no incluye todos los campos requeridos segun la validacion del index.php.	{"type":"error","msg":"Datos incompletos"}
401 Unauthorized	No autorizado	El header Authorization no fue enviado. v1.php detecta la ausencia y corta la ejecucion antes del modulo.	{"type":"error","msg":"Sin Autorizacion"}
403 Forbidden	Prohibido	El token fue enviado pero no coincide con el esperado para ese metodo HTTP especifico.	{"type":"error","msg":"Acceso Prohibido"}
422 Unprocessable	No procesable	La solicitud y el token son correctos, pero la consulta SQL no afecto ninguna fila (ID no existe en la BD).	{"type":"error","msg":"No se pudo crear el candidato"}
500 Internal	Error servidor	Excepcion no controlada en el servidor: fallo de conexion a la BD, error de sintaxis PHP, etc.	{"type":"error","msg":"Error interno del servidor"}
501 Not Impl.	No implementado	Se uso un metodo HTTP no soportado (HEAD, CONNECT, TRACE). El case default del switch retorna este codigo.	{"type":"error","msg":"Metodo No Implementado"}

La distincion entre 401 y 403 es importante: 401 significa que no se presento ninguna credencial (el header Authorization no existe), mientras que 403 significa que la credencial si se presento pero es incorrecta o no tiene permisos para esa operacion especifica.

9.5 Como usar Swagger UI

```
http://localhost/proviemplea/API/swagger-ui/dist/index.html
```

Paso	Accion detallada
1	Abrir el navegador y navegar a la URL de Swagger UI. Verificar que aparezcan los 3 modulos con sus endpoints.
2	Hacer clic en el boton "Authorize" (candado abierto, arriba a la derecha).
3	Ingresar SOLO el valor del token sin la palabra "Bearer" (ej: candidatos.get). Swagger la agrega automaticamente. Clic en "Authorize" y luego "Close".
4	Hacer clic sobre el endpoint deseado (ej: GET /CANDIDATOS/) para expandirlo.
5	Hacer clic en el boton "Try it out" (esquina superior derecha del endpoint expandido).
6	Para POST/PUT: seleccionar un ejemplo del dropdown "Examples" o editar el JSON manualmente.
7	Hacer clic en el boton azul "Execute". Swagger UI envia la solicitud al servidor.
8	Revisar la seccion "Server response": verificar el codigo HTTP y el body JSON.
9	Para el siguiente metodo con token diferente: clic en "Authorize", borrar el anterior, ingresar el nuevo y repetir.

10. Capturas de Pantalla - Swagger UI

En esta seccion insertar las capturas de pantalla de cada operacion ejecutada desde Swagger UI.
Capturar el bloque "Server response" que muestra el codigo HTTP y el body JSON de la respuesta.

10.1 Modulo CANDIDATOS - Swagger UI

Tokens: candidatos.get / candidatos.post / candidatos.put / candidatos.patch / candidatos.delete

GET /CANDIDATOS/ - Lista de candidatos (codigo 200)

The screenshot displays the Swagger UI interface for the GET /CANDIDATOS/ endpoint. The 'Curl' section shows the command: `curl -X 'GET' \ 'http://localhost/proviemplea/API/CANDIDATOS/' \ -H 'accept: application/json' \ -H 'Authorization: Bearer candidatos.get'`. The 'Request URL' is `http://localhost/proviemplea/API/CANDIDATOS/`. The 'Server response' section shows a status code of 200. The 'Response body' is a JSON array of five candidate objects. The 'Response headers' section lists various headers including access-control, content-type, date, keep-alive, server, and x-powered-by.

```
curl -X 'GET' \
'http://localhost/proviemplea/API/CANDIDATOS/' \
-H 'accept: application/json' \
-H 'Authorization: Bearer candidatos.get'
```

Request URL

http://localhost/proviemplea/API/CANDIDATOS/

Server response

Code Details

200

Response body

```
{
  "data": [
    {
      "id": "2",
      "titulo_perfil": "Contador Auditor",
      "descripcion": "Manejo de Excel avanzado, contabilidad general y tributaria",
      "anios_experiencia": "5",
      "activo": "1"
    },
    {
      "id": "3",
      "titulo_perfil": "Diseñador Grafico",
      "descripcion": "Dominio de Adobe Illustrator, Photoshop y Figma para proyectos digitales",
      "anios_experiencia": "2",
      "activo": "1"
    },
    {
      "id": "4",
      "titulo_perfil": "Desarrollador Web Junior",
      "descripcion": "Conocimientos en HTML, CSS y PHP basico",
      "anios_experiencia": "1",
      "activo": "1"
    },
    {
      "id": "5",
      "titulo_perfil": "Tecnico en Informatica",
      "descripcion": "Experiencia en soporte tecnico, redes y desarrollo web basico",
      "anios_experiencia": "3",
      "activo": "1"
    }
  ]
}
```

Response headers

```
access-control-allow-headers: Authorization,content-type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 815
content-type: application/json; charset=UTF-8
date: Sat, 30 May 2026 18:07:54 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

POST /CANDIDATOS/ - Crear candidato (codigo 201)

Curl

```
curl -X 'POST' \
  'http://localhost/proviemplea/API/CANDIDATOS/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer candidatos.post' \
  -H 'Content-Type: application/json' \
  -d '{
    "VCH_TITULO_PERFIL": "Tecnico en Informatica",
    "VCH_DESCRIPCION": "Experiencia en soporte técnico, redes y desarrollo web basico",
    "INT_ANIOS_EXPERIENCIA": 3
  }'
```

Request URL

http://localhost/proviemplea/API/CANDIDATOS/

Server response

Code	Details
201	Response body <pre>can't parse JSON. Raw result:
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 78

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 79

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 80
 {"type":"msg","msg":"Candidato creado correctamente"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 551 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:10:19 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

PUT /CANDIDATOS/ - Actualizar candidato (codigo 200)

Curl

```
curl -X 'PUT' \
  'http://localhost/proviemplea/API/CANDIDATOS/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer candidatos.put' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_CANDIDATO": 1,
    "VCH_TITULO_PERFIL": "tecnico en Informatica Senior",
    "VCH_DESCRIPCION": "Experiencia en soporte tecnico, redes, servidores y desarrollo web",
    "INT_ANIOS_EXPERIENCIA": 5
  }'
```

Request URL

http://localhost/proviemplea/API/CANDIDATOS/

Server response

Code	Details
200	Response body <pre>can't parse JSON. Raw result:
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 101

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 102

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 103

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php on line 104
 {"type":"msg","msg":"candidato actualizado correctamente"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 726 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:11:08 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

PATCH /CANDIDATOS/ - Toggle estado (codigo 200)

Curl

```
curl -X 'PATCH' \
  'http://localhost/proviewplea/API/CANDIDATOS/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer candidatos.patch' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_CANDIDATO": 1
  }'
```

Request URL

http://localhost/proviewplea/API/CANDIDATOS/

Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice:: Only variables should be passed by reference in C:\xampp\htdocs\proviewplea\API\CANDIDATOS\modeloCandidato.php on line 135
 {"type":"msg","msg":"Estado del candidato actualizado"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 222 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:12:51 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Min64) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

DELETE /CANDIDATOS/ - Eliminar candidato (codigo 200)}

Curl

```
curl -X 'DELETE' \
  'http://localhost/proviewplea/API/CANDIDATOS/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer candidatos.delete' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_CANDIDATO": 1
  }'
```

Request URL

http://localhost/proviewplea/API/CANDIDATOS/

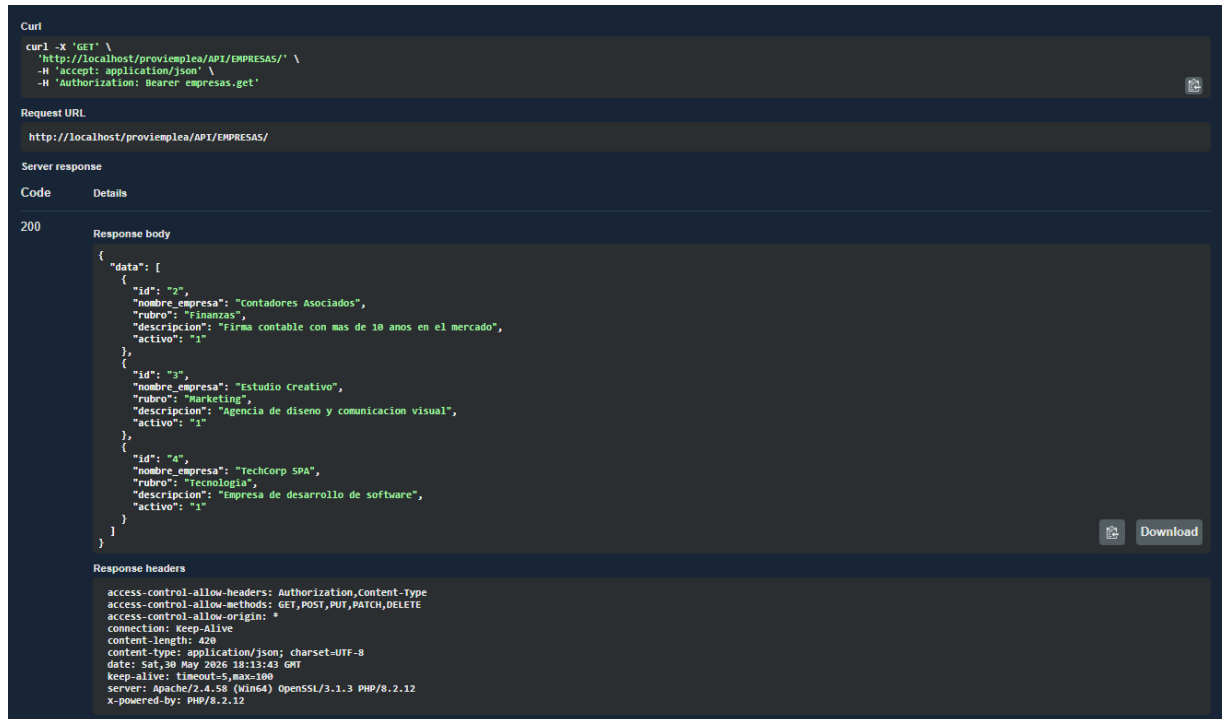
Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice:: Only variables should be passed by reference in C:\xampp\htdocs\proviewplea\API\CANDIDATOS\modeloCandidato.php on line 119
 {"type":"msg","msg":"Candidato eliminado correctamente"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 221 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:11:57 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Min64) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

10.2 Modulo EMPRESAS - Swagger UI

Tokens: empresas.get / empresas.post / empresas.put / empresas.patch / empresas.delete

GET /EMPRESAS/ - Lista de empresas (codigo 200)



The screenshot displays the Swagger UI interface for the GET /EMPRESAS/ endpoint. The 'Curl' tab shows the command: `curl -X 'GET' \ 'http://localhost/proviemplea/API/EMPRESAS/' \ -H 'accept: application/json' \ -H 'Authorization: Bearer empresas.get'`. The 'Request URL' is `http://localhost/proviemplea/API/EMPRESAS/`. The 'Server response' section shows a status code of 200. The 'Response body' is a JSON array of three company objects. The 'Response headers' section lists various headers including `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `connection`, `content-length`, `content-type`, `date`, `keep-alive`, `server`, and `x-powered-by`.

```
curl -X 'GET' \
'http://localhost/proviemplea/API/EMPRESAS/' \
-H 'accept: application/json' \
-H 'Authorization: Bearer empresas.get'
```

Request URL

http://localhost/proviemplea/API/EMPRESAS/

Server response

Code Details

200

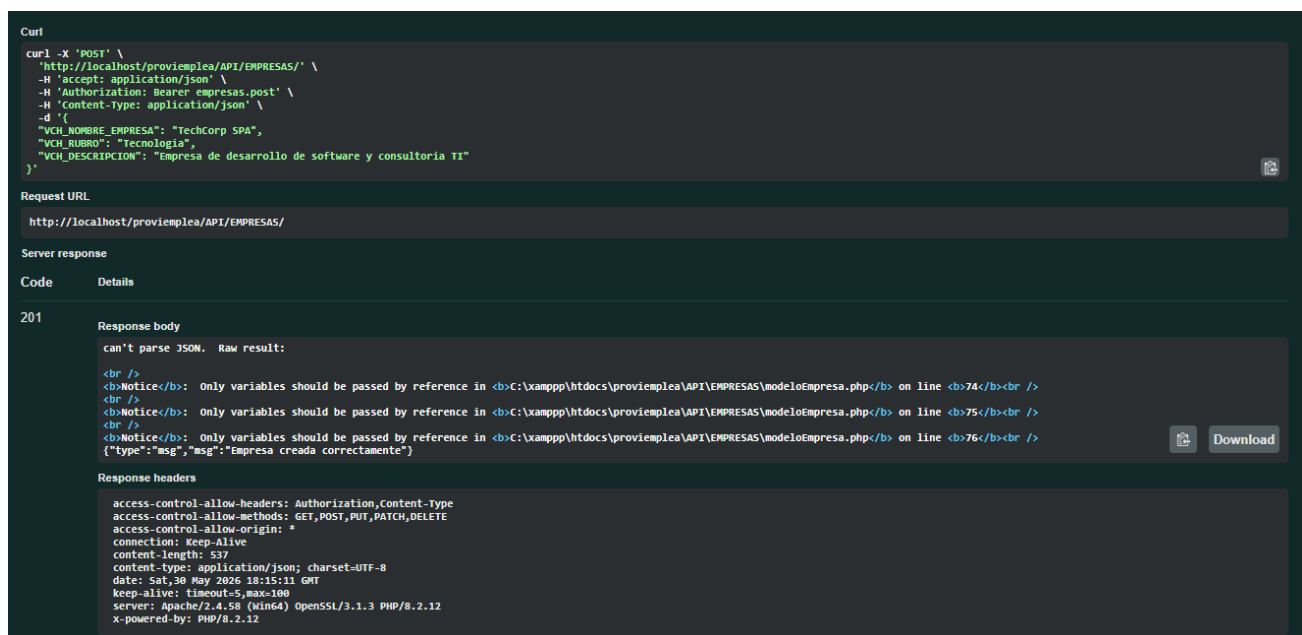
Response body

```
{
  "data": [
    {
      "id": "2",
      "nombre_empresa": "Contadores Asociados",
      "rubro": "Finanzas",
      "descripcion": "Firma contable con mas de 10 anos en el mercado",
      "activo": "1"
    },
    {
      "id": "3",
      "nombre_empresa": "Estudio Creativo",
      "rubro": "Marketing",
      "descripcion": "Agencia de diseno y comunicacion visual",
      "activo": "1"
    },
    {
      "id": "4",
      "nombre_empresa": "TechCorp SPA",
      "rubro": "Tecnologia",
      "descripcion": "Empresa de desarrollo de software",
      "activo": "1"
    }
  ]
}
```

Response headers

```
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 428
content-type: application/json; charset=UTF-8
date: Sat, 30 May 2026 18:13:43 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

POST /EMPRESAS/ - Crear empresa (codigo 201)



The screenshot displays the Swagger UI interface for the POST /EMPRESAS/ endpoint. The 'Curl' tab shows the command: `curl -X 'POST' \ 'http://localhost/proviemplea/API/EMPRESAS/' \ -H 'accept: application/json' \ -H 'Authorization: Bearer empresas.post' \ -H 'Content-type: application/json' \ -d '{ "VCH_NOMBRE_EMPRESA": "TechCorp SPA", "VCH_RUBRO": "Tecnologia", "VCH_DESCRIPCION": "Empresa de desarrollo de software y consultoria TI" }'`. The 'Request URL' is `http://localhost/proviemplea/API/EMPRESAS/`. The 'Server response' section shows a status code of 201. The 'Response body' contains a message indicating the company was created successfully, along with some raw JSON output. The 'Response headers' section lists various headers including `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `connection`, `content-length`, `content-type`, `date`, `keep-alive`, `server`, and `x-powered-by`.

```
curl -X 'POST' \
'http://localhost/proviemplea/API/EMPRESAS/' \
-H 'accept: application/json' \
-H 'Authorization: Bearer empresas.post' \
-H 'Content-type: application/json' \
-d '{
  "VCH_NOMBRE_EMPRESA": "TechCorp SPA",
  "VCH_RUBRO": "Tecnologia",
  "VCH_DESCRIPCION": "Empresa de desarrollo de software y consultoria TI"
}'
```

Request URL

http://localhost/proviemplea/API/EMPRESAS/

Server response

Code Details

201

Response body

```
can't parse JSON. Raw result:
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>74</b><br />
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>75</b><br />
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>76</b><br />
{"type":"msg","msg":"Empresa creada correctamente"}
```

Response headers

```
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 537
content-type: application/json; charset=UTF-8
date: Sat, 30 May 2026 18:15:11 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

PUT /EMPRESAS/ - Actualizar empresa (codigo 200)

Curl

```
curl -X 'PUT' \
  'http://localhost/proviemplea/API/EMPRESAS/' \
  -H 'accept: application/json' \
  -H 'authorization: Bearer empresas.put' \
  -H 'content-type: application/json' \
  -d '{
    "ID_EMPRESA": 1,
    "VCH_NOMBRE_EMPRESA": "TechCorp Chile SPA",
    "VCH_RUBRO": "Tecnología e Innovación",
    "VCH_DESCRIPCION": "Empresa de desarrollo de software, consultoría TI y transformación digital"
  }'
```

Request URL

http://localhost/proviemplea/API/EMPRESAS/

Server response

Code Details

200

Response body

can't parse JSON. Raw result:

```
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>97</b><br />
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>98</b><br />
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>99</b><br />
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>100</b><br />
{"type":"msg","msg":"Empresa actualizada correctamente"}
```

Response headers

```
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 795
content-type: application/json; charset=UTF-8
date: Sat,30 May 2026 18:15:58 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

PATCH /EMPRESAS/ - Toggle estado (codigo 200)

Curl

```
curl -X 'PATCH' \
  'http://localhost/proviemplea/API/EMPRESAS/' \
  -H 'accept: application/json' \
  -H 'authorization: Bearer empresas.patch' \
  -H 'content-type: application/json' \
  -d '{
    "ID_EMPRESA": 1
  }'
```

Request URL

http://localhost/proviemplea/API/EMPRESAS/

Server response

Code Details

200

Response body

can't parse JSON. Raw result:

```
<br />
<b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line <b>130</b><br />
{"type":"msg","msg":"Estado de la empresa actualizado"}
```

Response headers

```
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 210
content-type: application/json; charset=UTF-8
date: Sat,30 May 2026 18:17:21 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

DELETE /EMPRESAS/ - Eliminar empresa (codigo 200)

Curl

```
curl -X 'DELETE' \
  'http://localhost/proviemplea/API/EMPRESAS/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer empresas.delete' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_EMPRESA": 1
  }'
```

Request URL

```
http://localhost/proviemplea/API/EMPRESAS/
```

Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php on line 115
 {"type":"msg","msg":"Empresa eliminada correctamente"}</pre> Response headers<pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 217 content-type: application/json; charset=UTF-8 date: Sat, 30 May 2020 18:16:36 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

10.3 Modulo HABILIDADES - Swagger UI

Tokens: habilidades.get / habilidades.post / habilidades.put / habilidades.patch / habilidades.delete

GET /HABILIDADES/ - Catalogo de habilidades (codigo 200)

The screenshot displays the Swagger UI interface for the GET /HABILIDADES/ endpoint. The 'Curl' section shows the command: `curl -X 'GET' \ 'http://localhost/proviemplea/API/HABILIDADES/' \ -H 'accept: application/json' \ -H 'Authorization: Bearer habilidades.get'`. The 'Request URL' is `http://localhost/proviemplea/API/HABILIDADES/`. The 'Server response' section shows a status code of 200. The 'Response body' is a JSON array of six objects, each representing a skill with fields: id, nombre_habilidad, categoria, and activo. The 'Response headers' section lists various headers including access-control-allow-headers, access-control-allow-methods, access-control-allow-origin, connection, content-length, content-type, date, keep-alive, server, and x-powered-by.

```
Curl
curl -X 'GET' \
  'http://localhost/proviemplea/API/HABILIDADES/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer habilidades.get'

Request URL
http://localhost/proviemplea/API/HABILIDADES/

Server response
Code    Details
200

Response body
{
  "data": [
    {
      "id": "2",
      "nombre_habilidad": "MySQL",
      "categoria": "Tecnica",
      "activo": "1"
    },
    {
      "id": "3",
      "nombre_habilidad": "Trabajo en equipo",
      "categoria": "Blanda",
      "activo": "1"
    },
    {
      "id": "4",
      "nombre_habilidad": "Comunicacion efectiva",
      "categoria": "Blanda",
      "activo": "1"
    },
    {
      "id": "5",
      "nombre_habilidad": "Excel Avanzado",
      "categoria": "Tecnica",
      "activo": "1"
    },
    {
      "id": "6",
      "nombre_habilidad": "JavaScript",
      "categoria": "Tecnica",
      "activo": "1"
    }
  ]
}

Response headers
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 415
content-type: application/json; charset=UTF-8
date: Sat, 30 May 2026 18:18:10 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

POST /HABILIDADES/ - Crear habilidad (codigo 201)

The screenshot displays the Swagger UI interface for the POST /HABILIDADES/ endpoint. The 'Curl' section shows the command: `curl -X 'POST' \ 'http://localhost/proviemplea/API/HABILIDADES/' \ -H 'accept: application/json' \ -H 'Authorization: Bearer habilidades.post' \ -H 'content-type: application/json' \ -d '{ \"VCH_NOMBRE_HABILIDAD\": \"JavaScript\", \"VCH_CATEGORIA\": \"Tecnica\" }'`. The 'Request URL' is `http://localhost/proviemplea/API/HABILIDADES/`. The 'Server response' section shows a status code of 201. The 'Response body' is a JSON object with a message: `{\"type\": \"msg\", \"msg\": \"Habilidad creada correctamente\"}`. The 'Response headers' section lists various headers including access-control-allow-headers, access-control-allow-methods, access-control-allow-origin, connection, content-length, content-type, date, keep-alive, server, and x-powered-by.

```
Curl
curl -X 'POST' \
  'http://localhost/proviemplea/API/HABILIDADES/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer habilidades.post' \
  -H 'content-type: application/json' \
  -d '{
    \"VCH_NOMBRE_HABILIDAD\": \"JavaScript\",
    \"VCH_CATEGORIA\": \"Tecnica\"
  }'

Request URL
http://localhost/proviemplea/API/HABILIDADES/

Server response
Code    Details
201

Response body
{
  \"type\": \"msg\",
  \"msg\": \"Habilidad creada correctamente\"
}

Response headers
access-control-allow-headers: Authorization,Content-Type
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE
access-control-allow-origin: *
connection: Keep-Alive
content-length: 387
content-type: application/json; charset=UTF-8
date: Sat, 30 May 2026 18:19:14 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12
x-powered-by: PHP/8.2.12
```

PUT /HABILIDADES/ - Actualizar habilidad (codigo 200)

Curl

```
curl -X 'PUT' \
  'http://localhost/proviemplea/API/HABILIDADES/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer habilidades.put' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_HABILIDAD": 1,
    "VCH_NOMBRE_HABILIDAD": "PHP 8",
    "VCH_CATEGORIA": "Tecnica"
  }'
```

Request URL

http://localhost/proviemplea/API/HABILIDADES/

Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php on line 90

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php on line 91

 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php on line 92
 {"type":"msg","msg":"Habilidad actualizada correctamente"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 559 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:19:49 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

Download

PATCH /HABILIDADES/ - Toggle estado (codigo 200)

Curl

```
curl -X 'PATCH' \
  'http://localhost/proviemplea/API/HABILIDADES/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer habilidades.patch' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_HABILIDAD": 1
  }'
```

Request URL

http://localhost/proviemplea/API/HABILIDADES/

Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php on line 122
 {"type":"msg","msg":"Estado de la habilidad actualizado"}</pre> Response headers <pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 225 content-type: application/json; charset=UTF-8 date: Sat,30 May 2026 18:22:01 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

Download

DELETE /HABILIDADES/ - Eliminar habilidad (codigo 200)

Curl

```
curl -X 'DELETE' \
  'http://localhost/proviemplea/API/HABILIDADES/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer habilidades.delete' \
  -H 'Content-Type: application/json' \
  -d '{
    "ID_HABILIDAD": 1
  }'
```

Request URL
http://localhost/proviemplea/API/HABILIDADES/

Server response

Code	Details
200	Response body can't parse JSON. Raw result: <pre>
 Notice: Only variables should be passed by reference in C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php on line 107
 {"type":"msg","msg":"Habilidad eliminada correctamente"}</pre> Response headers<pre>access-control-allow-headers: Authorization,Content-Type access-control-allow-methods: GET,POST,PUT,PATCH,DELETE access-control-allow-origin: * connection: Keep-Alive content-length: 224 content-type: application/json; charset=UTF-8 date: Sat, 30 May 2026 18:20:50 GMT keep-alive: timeout=5,max=100 server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 x-powered-by: PHP/8.2.12</pre>

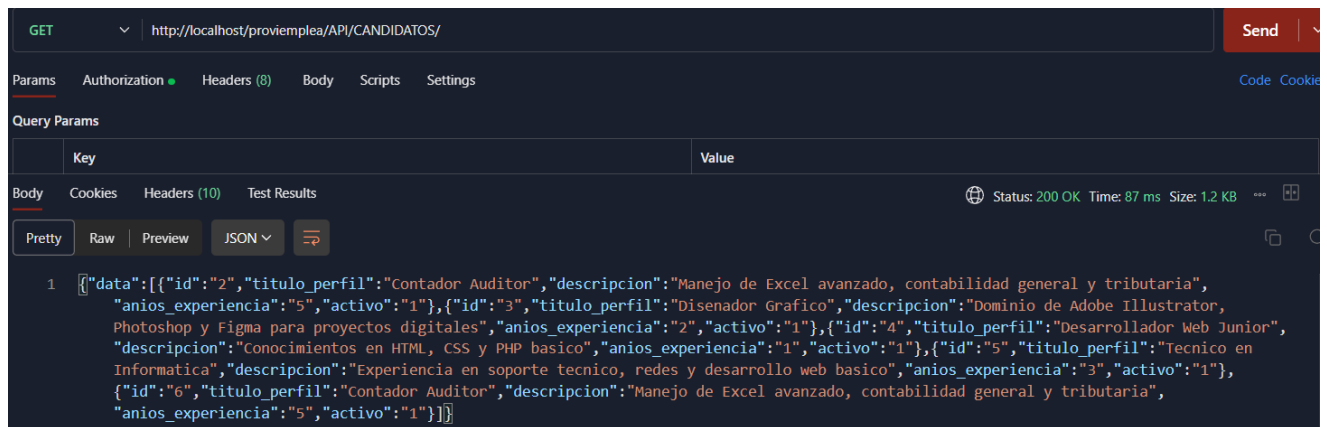
Download

11. Capturas de Pantalla - Postman

En esta seccion insertar las capturas de pantalla de cada operacion ejecutada desde Postman. Para cada captura, mostrar: el metodo HTTP, la URL, la configuracion del token en la pestana Auth y la respuesta recibida (status code + body).

11.1 Modulo CANDIDATOS - Postman

GET - Listar candidatos



POST - Crear nuevo candidato



PUT - Actualizar candidato

PUT http://localhost/proviemplea/API/CANDIDATOS/ Send

Params Authorization Headers (10) Body Scripts Settings

Body Cookies Headers (10) Test Results Status: 200 OK Time: 19 ms Size: 1.11 KB

Pretty Raw Preview JSON

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>101</b><br />
3 <br />
4 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>102</b><br />
5 <br />
6 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>103</b><br />
7 <br />
8 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>104</b><br />
9 {"type":"msg","msg":"Candidato actualizado correctamente"}
```

PATCH - Toggle estado activo/inactivo

PATCH http://localhost/proviemplea/API/CANDIDATOS/ Send

Params Authorization Headers (10) Body Scripts Settings

Query Params

Key	Value
-----	-------

Body Cookies Headers (10) Test Results Status: 200 OK Time: 7 ms Size: 636 B

Pretty Raw Preview JSON

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>135</b><br />
3 {"type":"msg","msg":"Estado del candidato actualizado"}
```

DELETE - Eliminar candidato

DELETE http://localhost/proviemplea/API/CANDIDATOS/ Send

Params Authorization Headers (10) Body Scripts Settings

Body Cookies Headers (10) Test Results Status: 200 OK Time: 8 ms Size: 637 B

Pretty Raw Preview JSON

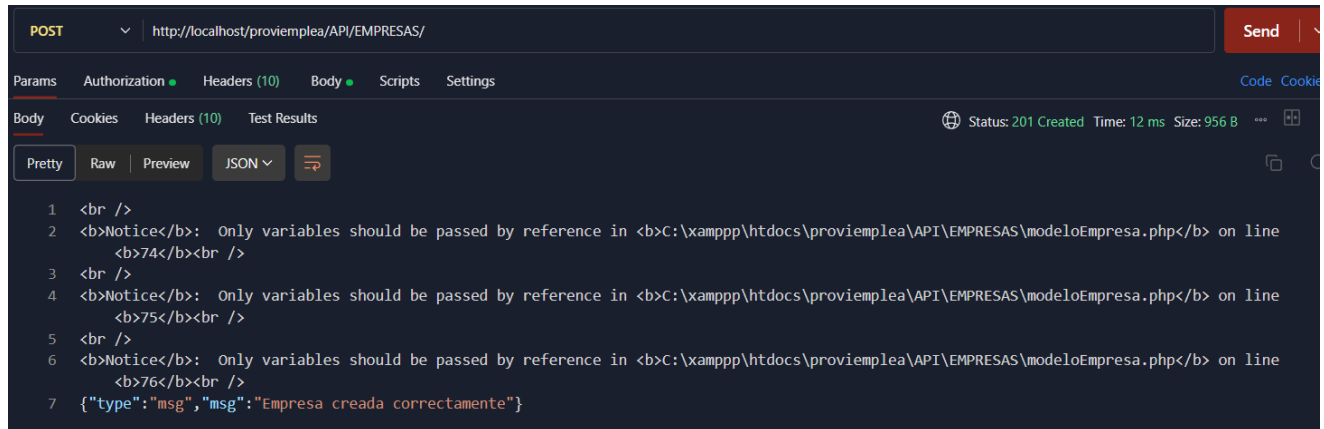
```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\CANDIDATOS\modeloCandidato.php</b> on
   line <b>119</b><br />
3 {"type":"msg","msg":"Candidato eliminado correctamente"}
```

11.2 Modulo EMPRESAS - Postman

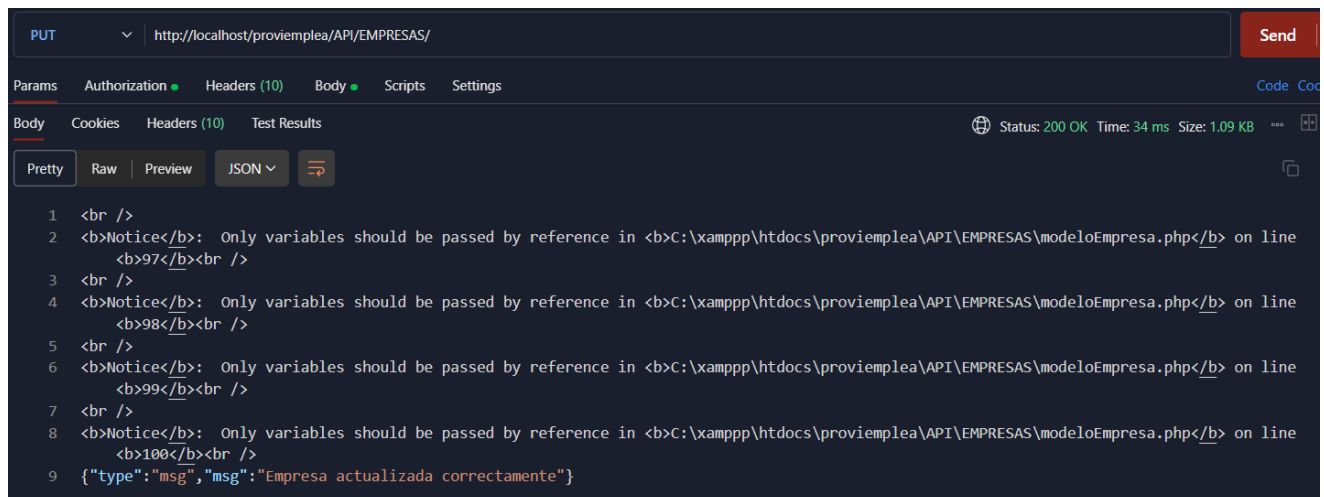
GET - Listar empresas



POST - Crear nueva empresa



PUT - Actualizar empresa



PATCH - Toggle estado activo/inactivo

PATCH ▼ http://localhost/proviemplea/API/EMPRESAS/ Send

Params Authorization ● Headers (10) Body ● Scripts Settings Code Coo

Body Cookies Headers (10) Test Results 🌐 Status: 200 OK Time: 7 ms Size: 632 B ... 🔍

Pretty Raw Preview JSON ▼ 🔗

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line
  <b>130</b><br />
3 {"type":"msg","msg":"Estado de la empresa actualizado"}
```

DELETE - Eliminar empresa

DELETE ▼ http://localhost/proviemplea/API/EMPRESAS/ Send

Params Authorization ● Headers (10) Body ● Scripts Settings Code Coo

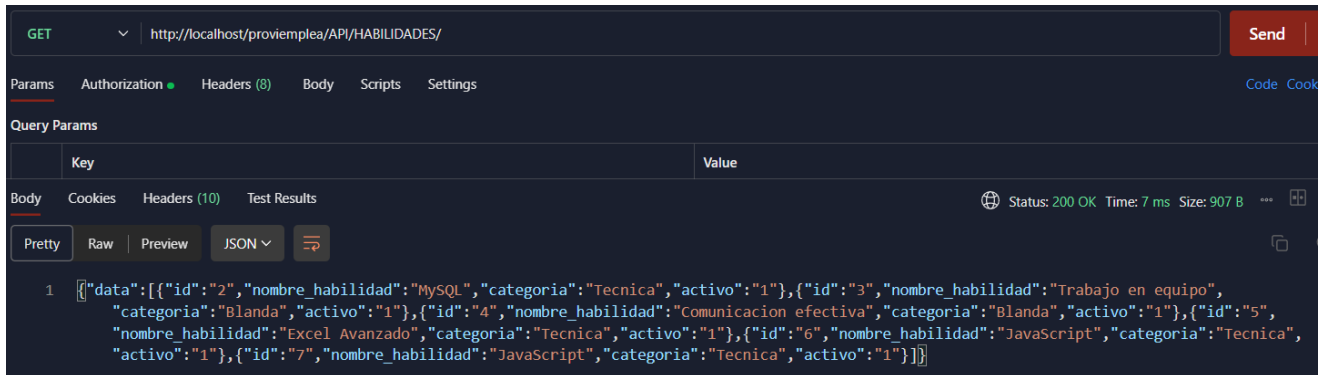
Body Cookies Headers (10) Test Results 🌐 Status: 200 OK Time: 10 ms Size: 631 B ... 🔍

Pretty Raw Preview JSON ▼ 🔗

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\EMPRESAS\modeloEmpresa.php</b> on line
  <b>115</b><br />
3 {"type":"msg","msg":"Empresa eliminada correctamente"}
```

11.3 Modulo HABILIDADES - Postman

GET - Catalogo de habilidades



Postman interface showing a GET request to `http://localhost/proviemplea/API/HABILIDADES/`. The response status is 200 OK, Time: 7 ms, Size: 907 B. The response body is a JSON array of skill objects.

```
1 [{"data":[{"id":"2","nombre_habilidad":"MySQL","categoria":"Tecnica","activo":"1"}, {"id":"3","nombre_habilidad":"Trabajo en equipo", "categoria":"Blanda","activo":"1"}, {"id":"4","nombre_habilidad":"Comunicacion efectiva","categoria":"Blanda","activo":"1"}, {"id":"5","nombre_habilidad":"Excel Avanzado","categoria":"Tecnica","activo":"1"}, {"id":"6","nombre_habilidad":"JavaScript","categoria":"Tecnica","activo":"1"}, {"id":"7","nombre_habilidad":"JavaScript","categoria":"Tecnica","activo":"1"}]}]
```

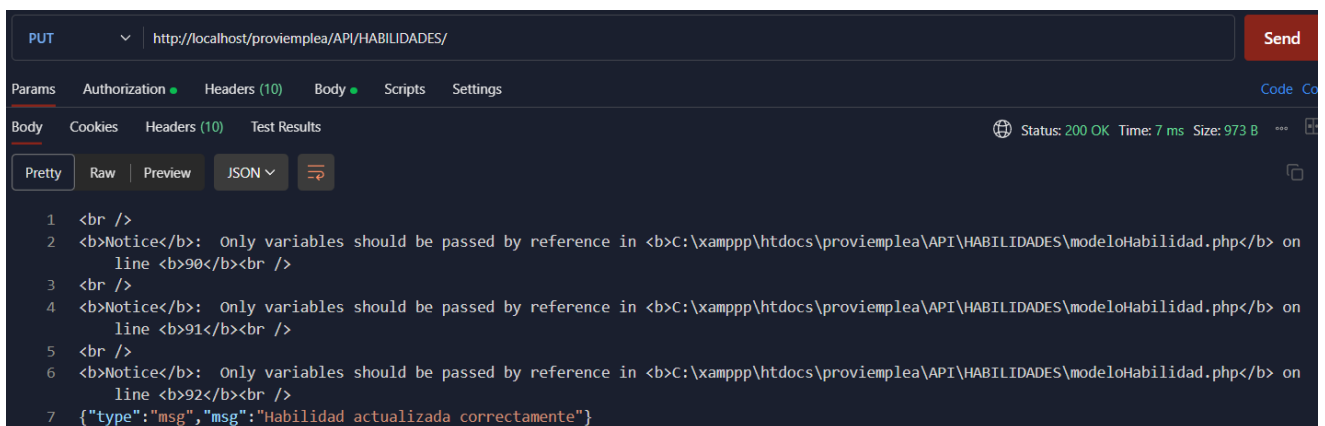
POST - Agregar habilidad



Postman interface showing a POST request to `http://localhost/proviemplea/API/HABILIDADES/`. The response status is 201 Created, Time: 9 ms, Size: 806 B. The response body is a JSON object with a message.

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>69</b><br />
3 <br />
4 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>70</b><br />
5 {"type":"msg","msg":"Habilidad creada correctamente"}
```

PUT - Actualizar habilidad



Postman interface showing a PUT request to `http://localhost/proviemplea/API/HABILIDADES/`. The response status is 200 OK, Time: 7 ms, Size: 973 B. The response body is a JSON object with a message.

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>90</b><br />
3 <br />
4 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>91</b><br />
5 <br />
6 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>92</b><br />
7 {"type":"msg","msg":"Habilidad actualizada correctamente"}
```

PATCH - Toggle estado activo/inactivo

PATCH

http://localhost/proviemplea/API/HABILIDADES/

Send

Params

Authorization

Headers (10)

Body

Scripts

Settings

Code

Co

Body

Cookies

Headers (10)

Test Results

Status: 200 OK Time: 6 ms Size: 639 B

Pretty

Raw

Preview

JSON

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>122</b><br />
3 {"type":"msg","msg":"Estado de la habilidad actualizado"}
```

DELETE - Eliminar habilidad

DELETE

http://localhost/proviemplea/API/HABILIDADES/

Send

Params

Authorization

Headers (10)

Body

Scripts

Settings

Code

Co

Body

Cookies

Headers (10)

Test Results

Status: 200 OK Time: 6 ms Size: 638 B

Pretty

Raw

Preview

JSON

```
1 <br />
2 <b>Notice</b>: Only variables should be passed by reference in <b>C:\xampp\htdocs\proviemplea\API\HABILIDADES\modeloHabilidad.php</b> on
   line <b>107</b><br />
3 {"type":"msg","msg":"Habilidad eliminada correctamente"}
```

12. Base de Datos - Estructura de Tablas

La base de datos se llama proviemplea y contiene tres tablas principales, una por cada modulo de la API. A continuacion se detalla la estructura SQL de cada tabla.

12.1 Tabla: tbl_candidato

```
CREATE TABLE tbl_candidato (      ID_CANDIDATO          INT(11)          NOT NULL
AUTO_INCREMENT,      VCH_TITULO_PERFIL          VARCHAR(100) NOT NULL,      VCH_DESCRIPCION
VARCHAR(500) NOT NULL,      INT_ANIOS_EXPERIENCIA INT(11)          NOT NULL DEFAULT 0,
INT_ACTIVO          INT(1)          NOT NULL DEFAULT 1,      PRIMARY KEY
(ID_CANDIDATO) ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

12.2 Tabla: tbl_empresa

```
CREATE TABLE tbl_empresa (      ID_EMPRESA          INT(11)          NOT NULL
AUTO_INCREMENT,      VCH_NOMBRE_EMPRESA VARCHAR(100) NOT NULL,      VCH_RUBRO
VARCHAR(100) NOT NULL,      VCH_DESCRIPCION          VARCHAR(300) NOT NULL,      INT_ACTIVO
INT(1)          NOT NULL DEFAULT 1,      PRIMARY KEY (ID_EMPRESA) ) ENGINE=InnoDB
DEFAULT CHARSET=utf8;
```

12.3 Tabla: tbl_habilidad

```
CREATE TABLE tbl_habilidad (      ID_HABILIDAD          INT(11)          NOT NULL
AUTO_INCREMENT,      VCH_NOMBRE_HABILIDAD VARCHAR(100) NOT NULL,      VCH_CATEGORIA
VARCHAR(20) NOT NULL COMMENT "Tecnica o Blanda",      INT_ACTIVO          INT(1)
NOT NULL DEFAULT 1,      PRIMARY KEY (ID_HABILIDAD) ) ENGINE=InnoDB DEFAULT
CHARSET=utf8;
```

Convenciones de nomenclatura de la BD

Prefijo	Tipo de dato	Ejemplo
ID_	INT - Clave primaria autoincremental	ID_CANDIDATO, ID_EMPRESA, ID_HABILIDAD
VCH_	VARCHAR - Texto de longitud variable	VCH_TITULO_PERFIL, VCH_NOMBRE_EMPRESA
INT_	INT - Numeros enteros	INT_ANIOS_EXPERIENCIA, INT_ACTIVO

13. Guia de Instalacion y Puesta en Marcha

Siga estos pasos en orden para dejar el proyecto operativo en un entorno local con XAMPP.

13.1 Requisitos previos

- XAMPP instalado con Apache y MySQL activos.
- PHP 7.4 o superior (incluido en XAMPP).
- Navegador web moderno (Chrome, Firefox, Edge).
- Postman (opcional, para pruebas adicionales a Swagger UI).

13.2 Pasos de instalacion

Paso	Accion
1	Copiar la carpeta proviemplea/ completa dentro de htdocs/ de XAMPP (normalmente C:\xampp\htdocs\ en Windows).
2	Abrir phpMyAdmin en http://localhost/phpmyadmin y crear una base de datos llamada proviemplea con cotejamiento utf8_general_ci.
3	Ejecutar los tres CREATE TABLE de la seccion 12 en la pestana SQL de phpMyAdmin para crear las tablas.
4	(Opcional) Insertar datos de prueba: ejecutar INSERT INTO tbl_candidato (VCH_TITULO_PERFIL, VCH_DESCRIPCION, INT_ANIOS_EXPERIENCIA) VALUES ("Tecnico en Redes", "Configuracion de routers y switches", 3);
5	Verificar que Apache este corriendo en XAMPP y abrir en el navegador: http://localhost/proviemplea/API/docs.php
6	Debe aparecer la interfaz de Swagger UI con los 3 modulos documentados. Si no carga, verificar que la carpeta swagger-ui/dist/ exista dentro de API/.
7	Probar el primer endpoint: clic en "Authorize", ingresar candidatos.get, clic en GET /CANDIDATOS/, "Try it out" y "Execute". Debe retornar codigo 200.

13.3 Solucion de problemas comunes

Problema	Causa probable	Solucion
Error 401 en todos los endpoints	El header Authorization no se esta enviando	Verificar que se hizo clic en "Authorize" en Swagger UI y se ingreso el token correcto.
Error 403 al ejecutar un metodo	El token es incorrecto para ese metodo HTTP	Cada metodo necesita su propio token (ej: candidatos.get para GET, candidatos.post para POST).
Swagger UI carga pero sin endpoints	El swagger.yaml no se encuentra en la ruta correcta	Verificar que swagger.yaml este en API/ y que la URL del parametro ?url= apunte correctamente.
Error de conexion a la BD	MySQL no esta corriendo o los datos de conexion son incorrectos	Verificar que el servicio MySQL este activo en el panel de XAMPP. Usuario: root, contrasena vacia.
Error 422 en POST correcto	La tabla no existe en la BD o el INSERT fallo silenciosamente	Verificar en phpMyAdmin que la tabla exista y tenga las columnas correctas.
La pagina docs.php muestra error 404	La ruta de Swagger UI es incorrecta	Verificar que la carpeta swagger-ui/dist/ exista y contenga el archivo index.html.